
FlowNAS: Guiding architectures optimization using Flow Matching

June 10, 2026

Alessandro Longo, Emanuela Tiberi

Abstract

Neural Architecture Search has received increasing attention in recent years as a way to automate the design of neural network architectures. In this project, we propose a NAS methodology that learns a direction of improvement in the latent architecture space learned by a VAE, guiding candidate architectures toward regions associated with higher accuracy.

1. Introduction & Related work

Neural Architecture Search (NAS) aims to automate the design of neural architectures through approaches ranging from evolutionary algorithms and reinforcement learning to continuous optimization. FlowNAS belongs to the latter category and learns a vector field in the latent architecture space, guiding initial architectures toward regions associated with higher performance.

Early methods for addressing NAS relied on evolutionary algorithms, such as NEAT (1), which evolves both network weights and topologies and inspired much of the subsequent research on evolutionary neural architecture search leading to interesting results, including AmoebaNet-A (2), which outperformed human-designed architectures for the first time, and (3), where the model *discovered* convolution. A major challenge in NAS is the discrete nature of the search space. To address this limitation, several methods reformulate the problem in a continuous space. DARTS (4) makes the search space differentiable and optimizes architectures through gradient descent. NAO (5), the method most closely related to ours, maps architectures into a learned continuous latent space, predicts their performance, and improves them by following the predictor’s gradient. FlowNAS differs by directly learning a vector field, thereby providing an explicit direction of improvement in the latent space.

Email: Alessandro Longo <Aless.longo03@gmail.com>, Emanuela Tiberi <tiberiemanuela03@gmail.com>.

Deep Learning and Applied AI 2026, Sapienza University of Rome, 2nd semester a.y. 2025/2026.

2. FlowNAS:

Ideally, we aim to map neural architectures into a continuous latent space in which architectures are organized according to their accuracy. To this end, we train the VAE using the following loss function:

$$\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \beta \mathcal{L}_{KL} + \lambda_{acc} \mathcal{L}_{acc}$$

where the last term is the mean squared error between the ground-truth accuracy and the predicted accuracy. The first panel of Figure 1 shows how architectures are organized by accuracy in the first two principal components of the learned latent space. After constructing the latent space, we aim to learn a direction of improvement. Let x_i denote the latent embedding of an architecture with accuracy y_i , and let x_j be the embedding of another architecture such that $y_j > y_i$. The flow-matching model is trained using the vector $x_j - x_i$ as the target direction. In this way, the model learns a vector field that moves architecture representations from lower-performing regions of the latent space toward regions associated with higher performance, as illustrated in the second panel of Figure 1.

Moving from the intuition to the actual procedure. The first stage of our algorithm is **VAE pretraining and freezing**: the VAE is pretrained on a subset of the dataset and its weights frozen. Once the VAE has been trained, N architectures are randomly generated from the search space and the following operations are performed:

- *Extract embeddings using pretrained VAE*: the N architectures are encoded using the pretrained VAE.
- *Build improvement pairs*: for each architecture embedding z_i , a random neighbor z_j with an accuracy higher by at least Δ is selected.
- *Train the flow*: for each improvement pair, the flow model is trained to predict the target direction $z_j - z_i$.
- *Generate new architectures using the flow*: once the flow has been trained, a new set of N architectures is generated by moving each latent representation along the learned vector field:

$$z_{k+1} = z_k + \eta v_\theta(z_k, t_k).$$

⁰<https://github.com/mvrморis/Deep-Learning-Project>

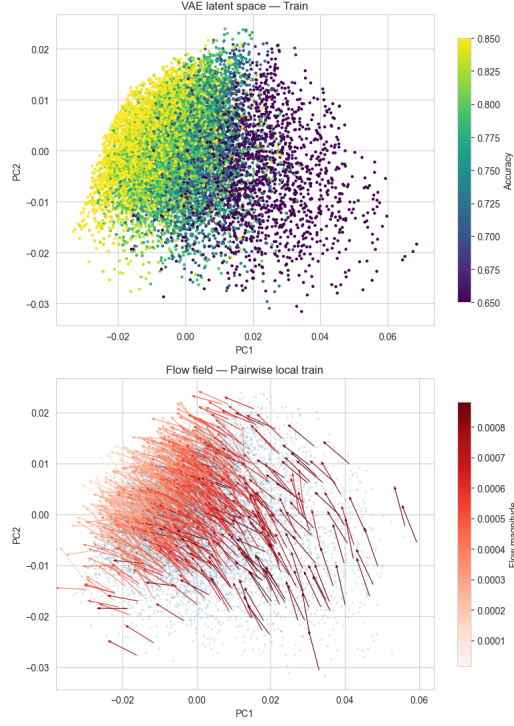


Figure 1. Inserisci qui la didascalia (opzionale)

- *Build the next generation*: the next generation is built by combining the architectures generated by the flow with a fraction of the best architectures (*elite fraction*) from the previous round. Duplicate architectures are removed.

Figure 2 shows why the elite fraction is needed. Across epochs, the accuracy distribution of the new population of architectures shifts toward better accuracies, this is the desired outcome. However, the distribution becomes narrower, indicating a gradual reduction in population diversity. Retaining an elite fraction helps preserve diversity and prevents the search from collapsing too early into a small set of architectures.

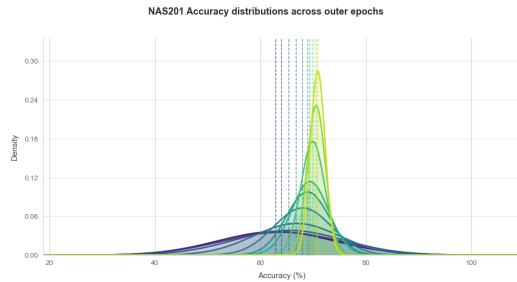


Figure 2. Accuracy distribution of architectures over epochs

2.1. Weight-sharing Supernet

Weight sharing allows different architectures to reuse the same trained parameters of a supernet, avoiding the need to train each architecture from scratch, the main computational bottleneck of NAS. The concept of a weight-sharing supernet was introduced by Pham et al. (6). A **Supernet** is modeled as a large directed acyclic graph (DAG) where nodes represent intermediate feature maps and edges represent candidate architectural operations, such as convolutions or pooling. Each path through the supernet $\mathcal{A}$ defines a specific architecture whose weights are shared with those of other architectures in the search space. The goal is to

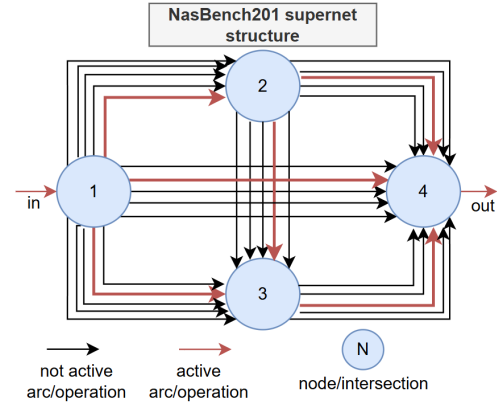


Figure 3. Example of the activation of a sampled path

optimize the global supernet weights W in a way that all subnet architectures are optimized. This multi-model optimization problem is formally expressed as

$$W^* = \arg \min_W \mathbb{E}_{a \sim \Gamma(\mathcal{A})} [\mathcal{L}_{\text{train}}(a; W)] \quad (1)$$

The shared weights W are optimized by minimizing the expected training loss of architectures a sampled from the distribution $\Gamma(\mathcal{A})$ over the search space $\mathcal{A}$. Pham et al. sampled architectures using an RNN controller trained with reinforcement learning. Guo et al. (7) later replaced this mechanism with uniform sampling in Single-Path NAS. They proposed sampling architectures uniformly from $\mathcal{A}$, making $\Gamma(\mathcal{A})$ a uniform distribution. Under this assumption, the gradient of the expected loss is approximated through Monte Carlo sampling by averaging the gradients computed over M sampled architectures:

$$\nabla_W \mathbb{E}_{a \sim \Gamma(\mathcal{A})} [\mathcal{L}_{\text{train}}(a; W)] \approx \frac{1}{M} \sum_{i=1}^M \nabla_W \mathcal{L}_{\text{train}}(a_i; W), \quad (2)$$

(typically $M = 1$ in standard stochastic gradient descent works fine in practice). Unlike SPOS (7), which trains

a single, massive supernet over the entire search space $\mathcal{A}$ just once, our approach introduces a **dynamic supernet** training strategy. Specifically, we use the supernet to evaluate each population by dynamically extracting a sub-supernet containing only the current candidate architectures. This dynamic supernet is then trained exclusively on these known, selected paths to extract their respective accuracies. The idea is that accuracy after this shorter training can approximate the final performance of each architecture. To validate this assumption, we measured Kendall’s τ between the accuracies estimated by the supernet and the ground-truth accuracies obtained after full training. The resulting correlation shown in Figure 4, shows that the relative ordering of the architectures is largely preserved, suggesting that the dynamic sub-supernet can provide a reliable architecture ranking and training the entire search space at once may not be necessary.

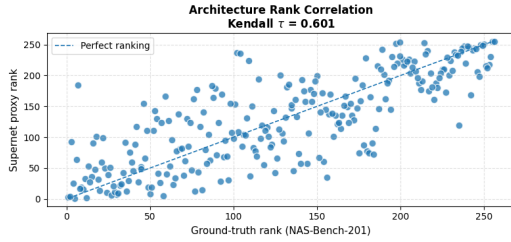


Figure 4. Correlation between long-trained network evaluation vs supernet evaluation

3. Experimental Results

To evaluate our approach and test each stage of the algorithm, we first used **NAS-Bench-201** (8), a lightweight benchmark with precomputed accuracies on CIFAR-10, CIFAR-100, and ImageNet16-120. To test the behavior of our algorithm on more complex search spaces, we then moved to the larger **NAS-Bench-301** (9), where CIFAR-10 performance is estimated using trained surrogate models. Both benchmarks adopt a similar approach: architectures are represented as directed acyclic graphs and each edge is associated with one operation selected from a predefined set, such as skip connections or convolutions.

For our experiments, we encoded each architecture using a tensor representation $A[\text{op}, \text{src}, \text{dst}]$. In particular, $A[i, j, k] = 1$ if operation i is assigned to the edge connecting source node j to destination node k , and $A[i, j, k] = 0$ otherwise. Accurate reconstruction is therefore crucial, since an incorrectly reconstructed value may result in a different operation being assigned to an edge. We initially trained the VAE using the loss described above, however, we observed that balancing the three loss terms did not produce sufficiently accurate reconstructions. We therefore set the coefficient of the KL-divergence term to zero, prioritizing reconstruction

quality. Future experiments could gradually increase this coefficient to investigate its potential regularization effect. The reconstruction loss of the NAS-Bench-301 VAE was also modified to account for the sparsity of the architecture representation. A standard reconstruction loss could encourage the model to predict mostly zeros while still achieving a low error. We therefore adopted a weighted reconstruction loss that assigns greater importance to correctly reconstructing the non-zero entries, which represent the actual operations.

Regarding the experiments on NAS-Bench-201, we observed that the final population generated by the algorithm contained the best architecture in the benchmark. Furthermore, Table 1 reports the comparison between the improvements obtained using the flow direction and a random direction. Across all three datasets, the flow direction outperforms the random baseline by at least 3%, indicating that the algorithm learns a meaningful direction of improvement in the latent space. Similar results are observed for NAS-Bench-301, although the improvement is smaller because most architectures are concentrated within a narrow accuracy range. The main hyperparameters used in the experiments are reported in Appendix 6.1.

Table 1. Comparison between the Flow direction and the random baseline on NAS-Bench-201, NAS-Bench-201 with weight sharing (WS), and NAS-Bench-301.

Benchmark	Dataset	Start	ΔF	ΔR	$\Delta(F - R)$
NAS-201	CIFAR-10	87.34%	+2.96%	-0.47%	+3.43%
NAS-201	CIFAR-100	61.67%	+4.27%	-0.53%	+4.79%
NAS-201	ImageNet16	33.75%	+4.63%	-0.48%	+5.11%
NAS-201 WS	CIFAR-10	+84.36%	+1.73%	-0.47%	+2.20%
NAS-201 WS	CIFAR-100	61.96%	+2.21%	-0.53%	+2.74%
NAS-301	CIFAR-10	93.10%	+0.27%	+0.0%	+0.27%

4. Conclusions

In this work, we presented an algorithm that learns a direction of improvement in a continuous latent space. While the learned direction consistently outperforms a random baseline, FlowNAS remains constrained by the predefined search unlike evolutionary algorithms, which can potentially discover new architectural patterns through mutation. Furthermore, our experimental evaluation was limited to a random-direction baseline. A more comprehensive assessment should compare FlowNAS with other NAS methods. Future work could also explore approaches that combine FlowNAS with evolutionary mechanisms to increase architectural diversity beyond the current elite-selection strategy. Finally, the model could be extended to optimize not only predictive performance but also architectural complexity in a multi-objective search.

5. Statement on the Use of AI

ChatGPT and Gemini were used as supporting tools throughout the development and writing of this work. Their use mainly concerned grammatical correction, rephrasing, stylistic revision, and the improvement of the clarity and readability of selected passages. These tools were also employed during the preliminary exploration of the Neural Architecture Search literature, helping us identify potentially relevant research directions and references. In addition, ChatGPT and Gemini assisted with the generation and refinement of code, particularly for experiments scripts, data processing and loading, result visualization, and plotting scripts. They were also used as debugging aids to identify possible implementation errors, interpret error messages, and suggest alternative solutions.

More broadly, these tools supported brainstorming and the iterative refinement of the proposed methodology. They were used to explore possible experimental settings, identify potential limitations of our approach, and further investigate our intuitions.

References

- [1] Kenneth O. Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002.
- [2] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019.
- [3] Chrisantha Fernando, Dylan Banarse, Malcolm Reynolds, Frederic Besse, David Pfau, Max Jaderberg, Marc Lanctot, and Daan Wierstra. Convolution by evolution: Differentiable pattern producing networks. *arXiv preprint arXiv:1606.02580*, 2016.
- [4] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations*, 2019.
- [5] Renqian Luo, Fei Tian, Tao Qin, Enhong Chen, and Tie-Yan Liu. Neural architecture optimization. In *Advances in Neural Information Processing Systems*, 2018.
- [6] Hieu Pham, Melody Guan, Barret Zoph, Quoc Le, and Jeff Dean. Efficient neural architecture search via parameter sharing. In *International conference on machine learning*, pages 4095–4104. PMLR, 2018.
- [7] Zichao Guo, Xiangyu Zhang, Haoyuan Mu, Wen Heng, Zechun Liu, Yichen Wei, and Jian Sun. Single path one-shot neural architecture search with uniform sampling. *arXiv preprint arXiv:1904.00420*, 2019.

Table 2. Main hyperparameters used for NAS-Bench-201 and NAS-Bench-301.

Parameter	NAS-201	NAS-301
Population size (N)	256	1024
Outer iterations	20	20
Latent dimension	16	32
VAE pretraining epochs	200	200
Pretraining fraction	0.5	0.5
β	0.0	0.0
λ_{acc}	1.0	1.0
Minimum improvement	0.01	0.001
Flow epochs	5	5
Flow step size (α)	0.5	0.5
Elite fraction	0.2	0.2

- [8] Xuanyi Dong and Yi Yang. Nas-bench-201: Extending the scope of reproducible neural architecture search. In *International Conference on Learning Representations*, 2020.
- [9] Julien N. Siems, Lucas Zimmer, Arber Zela, Jovita Lukasik, Margret Keuper, and Frank Hutter. Nas-bench-301 and the case for surrogate benchmarks for neural architecture search. *arXiv preprint arXiv:2008.09777*, 2020.

6. Appendix

6.1. Experiments Configuration

Table 2 reports the experimental settings. Due to the greater complexity and higher-dimensional input representation of NAS-Bench-301, the population size N , corresponding to the number of architectures considered at each iteration, was increased from 256 to 1024. The latent dimension was set to 32 instead of 16 for the same reason. Moreover, the minimum accuracy improvement used to construct training pairs was reduced from 0.01 to 0.001, since performance differences between NAS-Bench-301 architectures are generally smaller.